

Revision	Date	Updates	Approved By	Version_No.	Link to the SRS
Initial Version	January 20th 2026	Initial SRS document	Team	1.0	https://docs.google.com/document/d/179z_6r0txTU1sZZDwGoS8oVQpNvaKEDVpkiX-deas88/edit?usp=sharing
First Revision	January 21st 2026	Added REQ-TX-002 and REQ-TX-003.	Arnav	1.1	
		Added functional requirements and non-functional requirements.			
		Updated architecture to follow sharding pattern.			
Second Revision	January 24th 2026	Added REQ-REP-003 and REQ-REP-004 (failure detection and primary failover).	Team/ Reviewer Feedback	1.2	
		Added Requirements priorities.			
		Added Requirements priorities.			
		Naming conventions templates were documented.			
Third Revision	January 26th 2026	Added write-ahead logging requirements (REQ-DATA-002).	Subhanshu	1.3	
		Created the following new diagrams: timing, component.			
		Documented WAL recovery semantics to ensure atomicity.			
Fourth Revision	January 30th 2026	Added Kafka integration (REQ-COORD-001).	Anarghya	2.0	
		Mapped FR and NFR to sprints			
		Defined stateless coordinator model.			
Fifth Revision	February 4th 2026	Added load balancing requirements (REQ-DATA-003).	Team/ Reviewer Feedback	2.1	
		Revised the model for deadlock issue handling.			
		Documented shard rebalancing strategy.			
Sixth Revision	February 10th 2026	Added performance requirements (REQ-PERF-001 to REQ-PERF-004).	Arnav	2.2	
		Modified the following UML diagrams			
		Defined latency targets.			
Seventh Revision	February 15th 2026	Modified UML diagrams: Class, Activity and Use_Case.	Satwik	3.0	https://docs.google.com/document/d/1Ck8ca1uFEqezw1CbOK46mSsKxqEPLRHwMZtNrHETCml/edit?tab=t.0
Eight Revision	February 18th 2026	Updated Component Responsibilities.	Subhanshu	3.1	
Ninth Revision	March 18th 2026	Added Alpha Managers	Arnav Vivek	3.2	

Date	Diagram Category	Updates	Version_No.	Approved by	Link
13/2/26	Class Diagram	Separated single-shard and 2PC logic clearly. Added PartitionManager and Recovery inside ShardServer. Standardized transaction states and operation types.	2	Anarghya	https://drive.google.com/file/d/1EoslKbhcGZ2lPePICmoM5P1myJfyFkd/view?usp=sharing
15/2/26	Sequece Diagram	Added quorum acknowledgment. Aligned method names with class diagram.	2	Satwik	https://drive.google.com/file/d/1f1BHb8Nlc9siCkYD9WSaucUkUSGSf3A6/view?usp=sharing
19/2/26	Component Diagram	Organized into layered architecture. Added Engine abstraction for storage. Clarified WAL-based replication flow.	2	Arnav	https://drive.google.com/file/d/1M8tflPbyg3VONBKZJXwhBNKdjFPou3rO/view?usp=sharing
20/2/26	Composite Diagram	Expanded internal structure of API Gateway and Shard Node. Separated primary and follower responsibilities. Clarified internal wiring.	2	Satwik	https://drive.google.com/file/d/1Z4KQwj4O_Vs9Aw6m90wbjgAdLvqxpDOI/view?usp=sharing
24/2/26	Activity Diagram	Clearly separated single-shard and 2PC flows. Added decision points and crash recovery steps.	2	Subhanshu	https://drive.google.com/file/d/196j94jxRP5c2J07h_8NstKMCKEIAMxVj/view?usp=sharing
27/2/26	Use-Case Diagram	Refined actor roles. Added status polling and monitoring use cases. Clarified authentication linkage.	2	Anarghya	https://drive.google.com/file/d/1t9pCh5K7EylE58qcnRDdzcswlbAP99v6/view?usp=sharing
1/3/26	Timing Diagram	Separated single-shard and 2PC timelines. Highlighted prepare, commit, and replication phases.	2	Arnav	https://drive.google.com/file/d/1jUuFHI9jtB3PH_RvLxAE3b01ADHegb16/view?usp=sharing
2/3/26	Deployment Diagram	Separated API, coordinator, and shard nodes. Explicitly modeled primary, followers, and message broker.	2	Subhanshu	https://drive.google.com/file/d/1d5r1RRMVp3C2fFstfCv9643BEEo3cdb-/view?usp=sharing

Date	Alpha Concern	Alpha Card	Current Card	Status	Notes to support status	Approved by		Alpha Card	Alpha Manager
01/01/2026	Customer	Stakeholders	Recognized	1/6	Instructor and team identified as primary stakeholders.	Arnav Vivek		Stakeholders	Arnav Vivek
15/01/2026	Customer	Stakeholders	Represented	2/6	Roles and responsibilities agreed within team.	Arnav Vivek		Opportunity	Neeli Satwik
02/02/2026	Customer	Stakeholders	Involved	3/6	Instructor feedback incorporated during SRS v2.0 revision.	Arnav Vivek		Requirements	Anarghya Das
10/03/2026	Customer	Stakeholders	In Agreement	4/6	SRS v3.0 finalized with stakeholder alignment and feedback closure.	Arnav Vivek		Software System	Anarghya Das, Subhanshu Gupta
								Team	Arnav Vivek
05/01/2026	Customer	Opportunity	Identified	1/6	Need for scalable distributed ledger system defined.	Neeli Satwik		Work	Neeli Satwik
18/01/2026	Customer	Opportunity	Solution Needed	2/6	Sharding and 2PC selected as architectural approach.	Neeli Satwik		Way of Working	Subhanshu Gupta
02/02/2026	Customer	Opportunity	Value Established	3/6	Scope, constraints and objectives formalized in SRS v2.0.	Neeli Satwik			
12/03/2026	Customer	Opportunity	Viable	4/6	System demonstrates practical feasibility via working multi-shard execution.	Neeli Satwik			
10/01/2026	Solution	Requirements	Conceived	1/6	Initial functional requirements drafted.	Anarghya Das			
22/01/2026	Solution	Requirements	Bounded	2/6	Requirements categorized (TX, DATA, REP, COORD).	Anarghya Das			
02/02/2026	Solution	Requirements	Coherent	3/6	UML diagrams and traceability matrix completed.	Anarghya Das			
10/03/2026	Solution	Requirements	Acceptable	4/6	SRS v3.0 validated against implementation and test scenarios.	Anarghya Das			
25/01/2026	Solution	Software System	Architecture Selected	3/6	Gateway, Coordinator, Shard architecture finalized.	Anarghya Das; Subhanshu Gupta			
14/02/2026	Solution	Software System	Demonstrable	4/6	WAL-backed single-shard execution completed.	Anarghya Das; Subhanshu Gupta			
28/02/2026	Solution	Software System	Demonstrable	4/6	Multi-shard routing working end-to-end.	Anarghya Das; Subhanshu Gupta			
12/03/2026	Solution	Software System	Usable	5/6	Replication, failover, and recovery mechanisms integrated and working.	Anarghya Das; Subhanshu Gupta			
01/01/2026	Endeavor	Team	Seeded	1/5	Team members confirmed.	Arnav Vivek			
10/01/2026	Endeavor	Team	Formed	2/5	Sprint roles assigned.	Arnav Vivek			
20/02/2026	Endeavor	Team	Collaborating	3/5	Coordinated sprint execution across modules.	Arnav Vivek			
15/03/2026	Endeavor	Team	Performing	4/5	Efficient coordination during final integration and testing phase.	Arnav Vivek			
01/02/2026	Endeavor	Work	Initiated	1/6	Sprint 1 tasks started.	Neeli Satwik			
15/02/2026	Endeavor	Work	Prepared	2/6	Sprint 2 backlog defined.	Neeli Satwik			
02/03/2026	Endeavor	Work	Under Control	3/6	Sprint 3 started; cross-shard coordination in progress.	Neeli Satwik			
17/03/2026	Endeavor	Work	Converging	4/6	Sprint 3 nearing completion; replication, failover, and validation stabilized.	Neeli Satwik			
05/01/2026	Endeavor	Way of Working	Principles Established	1/6	Sprint structure and coding conventions defined.	Subhanshu Gupta			
20/01/2026	Endeavor	Way of Working	Foundation Established	2/6	Tools, repo structure, and workflow formally set up.	Subhanshu Gupta			
15/02/2026	Endeavor	Way of Working	In Use	3/6	Structured sprint workflow actively followed.	Subhanshu Gupta			
16/03/2026	Endeavor	Way of Working	Working Well	4/6	Team consistently follows practices with effective collaboration and delivery.	Subhanshu Gupta			

Requirement ID	Sprint ID	Component	Class	Functions	N. Start Date of Code	End Date of Code	Update Date	Update Made	Reason for Update	Team Member(s) Designing / Updating	Team members testing code	Completed/Ongoing/Aban	Ticket id/s raised	Link
REQ-TX-001	Sprint 1	API Gateway / Coordin	Transaction	NewTransaction, Validate, Submit	15/01/2026		10/02/2026	Added validation checks	Input correctness enforcement	Anarghya Das, Neeli Satwik	Neeli Satwik	Ongoing	TIC-TX-001	https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
REQ-TX-002	Sprint 1	Shard	Ledger	NewLedger, ApplyDebit, ApplyCredit, GetBalance, ValidateInvariants	15/01/2026	28/01/2026	05/02/2026	Bug fix in concurrent access	Thread-safety improvement	Neeli Satwik, Anarghya Das	Anarghya Das	Completed	TIC-TX-002-BUG	https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
REQ-TX-003	Sprint 3	Transaction Coordinat	Coordinator	Prepare, Commit, Abort, Execute2PC	15/02/2026		12/03/2026	Improved 2PC retry handling	Coordinator fault tolerance	Subhanshu Gupta, Anarghya Das	Neeli Satwik	Ongoing	TIC-TX-003	
REQ-TX-004	Sprint 3	Coordinator / Kafka	TransactionState	SetState, GetState, TransitionState	20/02/2026	08/03/2026	10/03/2026	Added missing state transitions	Consistency tracking	Amar Vivek	Subhanshu Gupta	Completed	TIC-TX-004	
REQ-DATA-001	Sprint 2	Shard Map Service	PartitionMapper	Hash_SetPartition, GetPartitionID	02/01/2026	10/02/2026	15/02/2026	Enhanced hash distribution	Load balancing improvement	Anarghya Das, Amav Vivek	Subhanshu Gupta	Completed	TIC-DATA-001	https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
REQ-DATA-002	Sprint 2	Shard Manager	PartitionManager	NewPartitionManager, OwnsPartition, AddPartition, RemovePartition	05/02/2026	14/02/2026	20/02/2026	Partition reassignment logic	Dynamic shard scaling	Anarghya Das	Anarghya Das	Completed		
	Sprint 1	Write Ahead Logging	WAL	NewWAL, Append, Fsync, ReadFrom, Truncate	20/01/2026	05/02/2026	20/02/2026	Added checkpointing support	Performance optimization	Amar Vivek, Neeli Satwik	Amar Vivek	Completed	TIC-DATA-002	https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
REQ-DATA-003	Sprint 1	WALEntry	WALEntry	NewWALEntry, Serialize, Deserialize	20/01/2026	25/01/2026	02/02/2026	Added serialization format	Structured logging	Amar Vivek	Subhanshu Gupta	Completed		https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
	Sprint 3	Shard Load Monitor	LoadBalancer	MonitorLoad, TriggerRebalance, MigratePartition	10/02/2026		15/03/2026	Added dynamic rebalancing	Runtime load handling	Amar Vivek, Subhanshu Gupta	Anarghya Das	Ongoing	TIC-DATA-003	
REQ-SAFE-001	Sprint 1	Ledger	Ledger	ValidateInvariant, GetBalance, SetBalance	08/02/2026	12/02/2026	18/02/2026	Added invariant checks	Prevent negative balances	Neeli Satwik	Neeli Satwik	Completed	TIC-SAFE-001	https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
REQ-SAFE-002	Sprint 3	Recovery	RecoveryEngine	Recover, ReplayCommitted, ValidateState, AccumulateBalances	01/02/2026		25/02/2026	Fixed zero-balance debit bug	Recovery correctness	Subhanshu Gupta, Anarghya Das	Amar Vivek	Ongoing	TIC-SAFE-002-BUG	
REQ-SAFE-003	Sprint 3	Recovery	WALRecovery	ReplayLog, RollbackUncommitted	15/02/2026	10/03/2026	14/03/2026	Improved rollback logic	Crash consistency	Subhanshu Gupta	Subhanshu Gupta	Completed	TIC-SAFE-003	https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
REQ-REP-001	Sprint 3	Replication	PrimaryReplicator	NewPrimaryReplicator, Replicate, SendToFollower	15/02/2026		06/03/2026	Optimized replication batching	Latency reduction	Subhanshu Gupta, Amav Vivek	Subhanshu Gupta	Ongoing	TIC-REP-001	https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
REQ-REP-002	Sprint 3	Replication	FollowerReceiver	NewFollowerReceiver, ReceiveEntry, PersistWAL, SendACK	18/02/2026		05/03/2026	ACK optimization	Quorum efficiency	Amar Vivek	Amar Vivek	Ongoing	TIC-REP-002	https://github.com/Subhanshu2405/Software-Course-Project/blob/0506853ebaf7c5ea091172674892a1689bc070/shard
REQ-REP-003	Sprint 3	Failure Detector	FailureDetector	SendHeartbeat, DetectFailure, NotifyReplicaManager	20/02/2026	12/03/2026	15/03/2026	Heartbeat tuning	Faster failure detection	Subhanshu Gupta	Amar Vivek	Completed	TIC-REP-003	
REQ-REP-004	Sprint 3	Replication Manager	FailureManager	ElectLeader, PromoteFollower, UpdateShardMap	01/03/2026		17/03/2026	Leader election logic	High availability groundwork	Subhanshu Gupta, Amav Vivek	Amar Vivek	Ongoing	TIC-REP-004	
REQ-COORD-001	Sprint 2	Kafka Log	KafkaProducer	Publish, EnsureOrdering, PersistOffset	15/02/2026		07/03/2026	Added partition ordering guarantee	Event consistency	Anarghya Das	Amar Vivek	Ongoing	TIC-COORD-001	
REQ-COORD-002	Sprint 3	Transaction Coordinat	RecoveryCoordinator	ResumeFromOffset, IdempotentExecute	01/03/2026		17/03/2026	Idempotent processing added	Failure recovery	Subhanshu Gupta	Subhanshu Gupta	Ongoing	TIC-COORD-002	
REQ-PERF-001	Sprint 3	Performance Monitor	MetricsCollector	TrackTPS, ReportMetrics	05/03/2026	17/03/2026		Performance instrumentation	System benchmarking	Amar Vivek	Subhanshu Gupta	Completed	TIC-PERF-001	
REQ-PERF-002	Sprint 3	Latency Tracker	LatencyMonitor	MeasureLatency, ComputeP99	05/03/2026	17/03/2026		Latency tracking	Performance validation	Amar Vivek	Neeli Satwik	Completed	TIC-PERF-002	
REQ-PERF-003	Sprint 3	Cross-Shard Optimizer	CoordinatorOptimizer	Optimize2PC, ReduceLatency	10/03/2026			Reduced coordination overhead	Latency improvement	Subhanshu Gupta	Anarghya Das	Ongoing	TIC-PERF-003	

Category	Expected Start Date	Expected End Date	 Actual Start Date	 Actual End Date	Percent Progression	Sprint Manager	Description
Project Planning	01/01/2026	14/01/2026	02/01/2026	14/01/2026	100%	Neeli Satwik	Project opportunity identification and value establishment. The initial requirements are finalized.
Requirements and SRS	14/01/2026	15/02/2026	15/01/2026	15/02/2026	75%	Anarghya Das	The requirements part will keep evolving in the agile methodology
Sprint 1	26/02/2026	04/03/2026	22/02/2026	06/03/2026	85%	Neeli Satwik	Establish the core system foundation: Database, Authentication and Sharding Logic.
Sprint 2	05/03/2026	11/03/2026	09/03/2026	15/03/2026	75%	Anarghya Das	Implement essential middleware such as WAL and pre-allocation features to handle the ACID conditions.
Sprint 3	12/03/2026	18/03/2026	16/02/2026	18/03/2026	65%	Subhanshu Gupta	Complete the fundamental load-balancing workflow and implement coordinator logic. Implement basic sharding requirements.
Sprint 4	19/03/2026	25/03/2026	18/03/2026	N/A	30%	Arnav Vivek	Integrate advanced sharding requirements.
Sprint 5	26/03/2026	04/01/2026	N/A	N/A	10%	Neeli Satwik	Implement advanced notification and UI/UX, and perform quality checks.
Sprint 6	04/02/2026	04/08/2026	N/A	N/A	35%	Arnav Vivek	Final System Validation
Final Review	04/09/2026	15/04/2026	N/A	N/A	0%	N/A	N/A

TEST TITLE	TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE	REQ-REP-002
WAL - Core	https://github.com/SAP/hana-sql-engine-2024-Software-Catalog/blob/main/qa/qa-test/wal/wal_test.go	High	TC-WAL-001	WAL-UNIT-01	01/25/2026	REQ-REP-004 election support
TEST DESCRIPTION	Unit tests for Write-Ahead Log core operations: append, commit markers, abort markers, monodonic log IDs, file creation, and empty-WAL behaviour.		TEST DESIGNED BY	TEST EXECUTED BY	EXECUTION DATE	REQ-REP-001 imply ACK
TEST DEPENDENCIES	shared/wal.go, shared/constants/tables.go, shared/models/wal_entry.go		Armin Vitek	Nadav Salach	01/25/2026	Development/impl-code setup
				TEST CONDITIONS		REQ-REP-001 failure path
				Fresh binary directory per test, no WAL file created on Open(); typic enabled		Each test opens a clean WAL, at REQ-REP-002 ordered log entries
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
WAL-01	TestSuccessfulInsert - append DEBT and CREDIT entries, next back and verify TxId, OpType, AccountID, Amount, sequential LogIDs	01/25/2026	2 entries returned, LogID 0-DEBT alicia 100; LogID 1 - entries, LogID=0 correct, all fields match	PASS	REQ-DATA-003	2PC global abort
WAL-02	TestMarkCommitted - write DEBT+COMMIT then MarkCommitted; check last entry is OpType=COMMITTED with correct TxId	01/25/2026	3 entries, last entry OpType=COMMITTED, TxId=0-3 entries, last entry COMMITTED, TxId=0 matches	PASS		Log before apply rule
WAL-03	TestMarkAborted - write DEBT then MarkAborted; verify last entry OpType=ABORTED	01/25/2026	Last WAL entry has OpType=ABORTED	Last entry ABORTED confirmed	PASS	REQ-REP-003 liveness
WAL-04	TestPersistAndReplay - write 3 entries, close, reopen; entries survive and NextLogID resumes at 3	01/25/2026	3 entries after reopen; NextLogID=3 (no ID collision)	3 entries recovered; NextLogID=3	PASS	Crash safety simulation
WAL-05	TestNextLogIDMonotonicallyIncreases - append 10 entries, each LogID strictly greater than previous	01/25/2026	LogIDs 0-9 strictly monotonic increasing	ID=0 monotonic, no repeats	PASS	REQ-DATA-002
WAL-06	TestWALFileExistsAfterWrite - append one entry, close; verify file exists on disk	01/25/2026	WAL file present on filesystem after write to close	File exists at expected path	PASS	REQ-TX-001
WAL-07	TestEmptyWALRead - ReadAll on brand new WAL with no entries	01/25/2026	Returns empty alicia, no error	Empty alicia returned, no err	PASS	REQ-TX-002
...	...					REQ-DATA-002
						REQ-COORD-002 entry safety
						REQ-DATA-003, Algorithm 5
						Atomicity guarantee
TEST TITLE	TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE	
WAL - Checkpoint & Compaction	https://github.com/SAP/hana-sql-engine-2024-Software-Catalog/blob/main/qa/qa-test/wal/wal_checkpoint_test.go	High	TC-WAL-002	WAL-UNIT-02	02/12/2026	
TEST DESCRIPTION	Unit tests for WAL checkpoint markers, partial replay (ReadFrom), truncation (compact), and the full checkpoint - truncate workflow cycle.		TEST DESIGNED BY	TEST EXECUTED BY	EXECUTION DATE	
TEST DEPENDENCIES	shared/wal.go, shared/constants/tables.go (CHECKPOINT)		Armin Vitek	Nadav Salach	02/12/2026	
				TEST CONDITIONS		
				Fresh WAL, per test; checkpoint writes treated as regular typed entries		All paths use 1 TempDir; WAL closed in Cleanup()
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
WALCK-01	TestWALWriteCheckpoint - append DEBT+COMMIT, MarkCommitted, WriteCheckpoint; last entry must be OpCheckpoint with correct CheckpointLogID	02/12/2026	Last entry OpType=CHECKPOINT; CheckpointLogID entry present; CheckpointLogID correct	PASS		Column 1
WALCK-02	TestWALReadFrom - write 4 entries (IDs 0-4), ReadFrom(0) should return exactly 2 entries (IDs 3,4)	02/12/2026	ReadFrom(0) returns 2 entries; all have LogID == 3	2 entries returned; LogIDs 3 and 4	PASS	REQ-DATA-003
WALCK-03	TestWALTruncate - write 4 entries (IDs 0-3), Truncate(0); only IDs 4,5 remain; new append succeeds	02/12/2026	2 entries remain after truncate; first entry LogID=4	2 entries; LogID=4 post-truncate append OK	PASS	Partial replay after checkpoint
WALCK-04	TestWALTruncateAndWrite - write 2 entries, Truncate(0); WAL should be empty	02/12/2026	0 entries after truncate with very high beforeLogID	0 entries confirmed	PASS	Atomic rename used
WALCK-05	TestWALCheckpointAndTruncateWorkflow - full cycle: commit tx=1, record checkpoint pos, commit tx=2, WriteCheckpoint, Truncate; no entries with LogID > checkpoint pos remain	02/12/2026	All remaining entries have LogID == checkpoint pos	No pre-checkpoint entries in WAL after truncate	PASS	Edge case
						Full snapshot semantics
						Full compaction cycle
						REQ-DATA-001
...	...					
						Load distribution
						Full test config guard
						Even distribution
TEST TITLE	TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE	
Ledger	https://github.com/SAP/hana-sql-engine-2024-Software-Catalog/blob/main/qa/qa-test/ledger/ledger_test.go	High	TC-LED-001	LED-UNIT-01	01/25/2026	REQ-DATA-001
TEST DESCRIPTION	Unit tests for Ledger account management and balance operations: account creation, negative balance invariant, atomic transfers, conservation of money, rollback, snapshot creation, and edge cases.		TEST DESIGNED BY	TEST EXECUTED BY	EXECUTION DATE	Migration support
TEST DEPENDENCIES	shared/ledger.go		Nadav Salach	Nadav Salach	01/25/2026	Crash-safe file update
				TEST CONDITIONS		
				Fresh in-memory ledger per test, no WAL interaction, mutex correctness tested implicitly		All tests construct isolated ledger instances; no shared state between cases
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
LED-01	TestCreateAccountAndGetBalance - create alicia with balance 1000; GetBalance returns (1000, true)	01/25/2026	GetBalance(alicia) == (1000, true)	(1000, true) returned	PASS	REQ-DATA-002
LED-02	TestCreateAccountNegativeBalanceFails - CreateAccount with opening balance -1 must return error	01/25/2026	Error returned for negative opening balance	Error returned; account not created	PASS	No-negative invariant
LED-03	TestCreateDuplicateAccountFails - create alicia twice; second call must error	01/25/2026	Second CreateAccount returns non-nil error	Error returned on duplicate	PASS	Migration Algorithm 6
LED-04	TestApplyTransferSuccess - alicia=1000, bob=500; transfer 300; alicia=700, bob=800; total conserved	01/25/2026	alice=700, bob=800, TotalBalance unchanged	Balance correct; conservation holds	PASS	Atomic debit-credit
LED-05	TestApplyNegativeBalance - alicia=100, attempt transfer 200; must fail, alicia unchanged at 100	01/25/2026	Error returned; also balance stays 100	Error returned; alicia=100	PASS	Critical invariant
LED-06	TestApplyTransferZeroAmountFails - attempt transfer of amount=0; must return error	01/25/2026	Error returned for zero amount transfer	Error returned	PASS	
LED-07	TestApplyTransferUnknownSource - source account 'ghost' doesn't exist; must error	01/25/2026	Error for unknown source account	Error returned	PASS	Idempotency
LED-08	TestApplyTransferUnknownDest - destination 'ghost' doesn't exist; must error	01/25/2026	Error for unknown destination account	Error returned	PASS	
LED-09	TestConservationOfMoneyMultipleTransfers - 5 sequential transfers among 3 accounts; TotalBalance = 3000 before and after	01/25/2026	TotalBalance = 3000 throughout all 5 transfers	Conservation holds; total=3000	PASS	Invariant: conservation of value
LED-10	TestValidateAndCommitChangeBalance - ValidateAndCommit(alicia, 200) must be side-effect free; also balance unchanged	01/25/2026	also balance = 500 after ValidateAndCommit	Balance unchanged at 500	PASS	REQ-DATA-002
LED-11	TestRollbackAndCommit - ApplyAndCommit 200 from alicia=500 -> 300; RollbackAndCommit -> restored to 500	01/25/2026	After rollback, alicia=500	alice=500 after rollback	PASS	2PC PREPARE phase relies on 1 No-negative invariant
LED-12	TestSnapshotIsolation - Snapshot returns copy; mutating copy must not affect live ledger	01/25/2026	alice live balance stays 500 after copy is modified	live ledger unaffected; snapshot isolated	PASS	2PC ABORT path
LED-13	TestNewLedgerWithAccountsNegativeFails - construct ledger with bob=-50; must return error	01/25/2026	Constructor returns non-nil error for negative initial	Error returned	PASS	Partition migration safety
						Atomic debit-credit
						Critical invariant
...	...					
						Invariant: conservation of value
						2PC PREPARE phase relies on this
						Partition migration safety
TEST TITLE	TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE	
Partition Manager	https://github.com/SAP/hana-sql-engine-2024-Software-Catalog/blob/main/qa/qa-test/partition/partition_test.go	Medium	TC-PART-001	PART-UNIT-01	03/15/2026	REQ-DATA-003
TEST DESCRIPTION	Unit tests for shard Partition Manager: initial ACTIVE state, ownership queries, ha/restore lifecycle, dynamic add/remove, active-partition filtering, idempotent halt, and ShardID accessor		TC-PART-001	PART-UNIT-01	03/15/2026	Partial replay after checkpoint
TEST DEPENDENCIES	shared/partition/partition.go		Anarghya Das	Anarghya Das	02/15/2026	Atomic rename used
				TEST CONDITIONS		
				In-memory partition state only; no WAL or network dependency		Fresh Manager constructed per / Edge case
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
PART-01	TestNewManagerActiveState - create Manager with partitions (1,2); all must be owned AND active	02/15/2026	OwnedPartitions(1,2)=true, IsActive(1,2)=true	All 3 owned and active	PASS	Initial state
PART-02	TestOwnerPartitionsAndOwnership - partition 0 is not in initial set; OwnerPartitions(0) returns false	02/15/2026	Both return false for partition 0	false for both	PASS	REQ-DATA-003
PART-03	TestHaltAndResumePartition - halt partition 1; owned but not active; other partitions unaffected; resume -> active again	02/15/2026	After halt: owner=true, active=false for p1; p1 still as 1; cycle correct; p0 unaffected	1; cycle correct; p0 unaffected	PASS	Log before apply rule
PART-04	TestHaltOnOwnedPartitionErrors - HaltPartitions(0) must return error	02/15/2026	Non-nil error for halting unowned partition	Error returned	PASS	Migration Algorithm 6
PART-05	TestAddAndRemovePartition - AddPartitions() -> active; RemovePartitions() -> not owned	02/15/2026	p5 active after add; p0 not owned after remove	Dynamic management works	PASS	Crash safety simulation
PART-06	TestOwnedPartitions - create with (3,5,7); OwnedPartitions() returns slice of length 3	02/15/2026	OwnedPartitions() length = 3	Length=3	PASS	Crash safety simulation
PART-07	TestActivePartitionsAndIsOwned - halt partition 1 from (1,2); ActivePartitions() has 2 elements; p1 owned	02/15/2026	ActivePartitions() = 2 items; partition 1 not in list	2 active; p1 excluded	PASS	Atomic rename used
PART-08	TestHaltIdempotent - HaltPartitions(0) twice; second call must return error	02/15/2026	Second halt returns nil error	No error on double halt	PASS	Idempotency
PART-09	TestShardID - Manager created with shardID=shard-42; ShardID() returns shard-42	02/15/2026	ShardID() = shard-42	shard-42 returned	PASS	
...	...					
TEST TITLE	TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE	
Shard Map	https://github.com/SAP/hana-sql-engine-2024-Software-Catalog/blob/main/qa/qa-test/shard_map/shard_map_test.go	High	TC-SM-001	SM-UNIT-01	02/20/2026	
TEST DESCRIPTION	Unit tests for ShardMap: file creation, round-robin partition distribution, account-to-shard resolution, partition update, ZJOIN persistence across reload, ASHards deduplication, and empty-shard error		TEST DESIGNED BY	TEST EXECUTED BY	EXECUTION DATE	
TEST DEPENDENCIES	coord/shardmap/shardmap_test.go, shared/shardmap.go		Anarghya Das, Armin Vitek	Anarghya Das	02/20/2026	
				TEST CONDITIONS		
				Temp dir for JSON file; 2-3 shards; partition counts divisible by shard count for round-robin tests		Fresh JSON file per test; file removed with 1 TempDir cleanup
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
SM-01	TestNewShardMapOutOfFile - create map with 2 shards and 5 partitions; PartitionCount=4; JSON file exists on disk	02/20/2026	PartitionCount(4); file state returns no error	Count=4; file present	PASS	Column 1
SM-02	TestShardMapRoundRobinDistribution - 3 shards, 9 partitions; each shard owns exactly 3 partitions	02/20/2026	GetPartitionsForShard() -> 3 for each of 3 shards	Each shard owns 3 partitions	PASS	Even distribution
SM-03	TestShardMapPartialShardAccount - resolve 'shard-0' to a shard; result is one of the known shards; deterministic on re-call	02/20/2026	ShardID in (shard-1, shard-2); same result on second call	Deterministic shard resolution	PASS	REQ-DATA-001
SM-04	TestShardMapReassignPartitions - reassign partition 0 to shard-3; GetShardID() ShardID=shard-3	02/20/2026	GetShardID() returns shard-3 after update	Partition 0 reassigned correctly	PASS	Migration support
SM-05	TestShardMapPersistence - create 5-partition map; reload from file; PartitionCount()=5 after reload	02/20/2026	Reloaded map has 5 partitions	Persistence works across LoadShardMap	PASS	Crash-safe file update

SM-06	TestShardMapAllShards – 2 shards, 6 partitions; AllShards() returns deduplicated slice of length 2		AllShards() length = 2		Develop correct; length=2	02/20/2026	PASS		
SM-07	TestShardMapNoShardError – NewShardMap with empty shard list must return error		Non-nil error returned		Error returned	02/20/2026	PASS		
*** *** ***									
TEST TITLE		TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE			
Hash / Partition Mapper		https://github.com/SatyaPusha-256-Software-Corpus/Corpus/blob/main/Tests/HashMapper_test.go	High	TC-HASH-001	HASH-UNIT-01	01/20/2026			
TEST DESCRIPTION		Unit tests for SHA-256 PartitionMapper: deterministic, range validation, isSamePartition consistency, uniform distribution (±0.05 tolerance with 3000 accounts), PartitionKey format, and panic on zero partitions.				TEST EXECUTED BY	ANAND DAS	EXECUTION DATE	01/20/2026
TEST DEPENDENCIES		shaveutils/hash.go				TEST CONDITIONS	30 partitions default, 3000 accounts for distribution test, SHA-256 hash function	TEST CONTROL	(Balance mapper; no external dependencies; all tests use NewPartitionMapper)
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES	Column 1		
HASH-01	TestGetPartitionsDeterministic – call GetPartition(alice) 100 times; all calls must return the same value	01/20/2026	All 100 calls return identical partition ID	Deterministic across 100 calls	PASS	REQ-DATA-001			
HASH-02	TestGetPartitionsInRange – 1000 distinct accounts (0x); all partition IDs in [0, 30]	01/20/2026	Every partition ID satisfies 0 ≤ id < 30	All 1000 in range	PASS				
HASH-03	TestIsSamePartitionConsistency – for all pairs in (alice bob, charlie, dave); isSamePartition(a,b) must agree with GetPartition(a) == GetPartition(b)	01/20/2026	isSamePartition matches manual comparison for a ± 25 pairs consistent		PASS				
HASH-04	TestDistributionUniformityUniform – 3000 accounts across 30 partitions; each partition within ±0.05 of expected 100	01/20/2026	Each of 30 partitions has count in [40, 160]	All partitions within tolerance	PASS	Load distribution			
HASH-05	TestPartitionKeyFormat – PartitionKey(alice) returns string matching 'partition-id-format'	01/20/2026	Key equals 'partition-id-convo(GetPartition(alice))'	Format correct	PASS				
HASH-06	TestNewPartitionMapperPanicOnZero – NewPartitionMapper(0) must panic	01/20/2026	recover() catches non-nil panic	Panic recovered	PASS	Fail-fast config guard			
*** *** ***									
TEST TITLE		TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE			
Storage Engine (JSONB)		https://github.com/SatyaPusha-256-Software-Corpus/Corpus/blob/main/Tests/StorageEngine_test.go	Medium	TC-STG-001	STG-UNIT-01	02/25/2026			
TEST DESCRIPTION		Unit tests for the JSONB persistent storage engine: file creation, set/get balance, missing account, batch set, checkpoint log ID, cross-dataset persistence, and batch overwrites semantics.				TEST EXECUTED BY	ANAND DAS	EXECUTION DATE	02/25/2026
TEST DEPENDENCIES		storageengine.go, storageengine_test.go				TEST CONDITIONS	Temp directory per test; JSON file created on first SetBalance write	TEST CONTROL	newTestStore() helper wires 1 Cleanup(Close); separate path per test prevents cross-contamination
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES	Column 1		
STG-01	TestNewJSONStoreCreateFile – SetBalance(alice, 1000); verify JSON file exists on first write	02/25/2026	vs StorePath() returns no error after first write	File present	PASS				
STG-02	TestJSONBatchGetSetGetBalance – SetBalance(alice, 1000); GetBalance(1000, true, nil)	02/25/2026	batch=1000; exists=true; err=nil	Correct values returned	PASS				
STG-03	TestJSONBatchGetNonExistentAccount – GetBalance(nobody) on empty store returns exists=false	02/25/2026	exists=false; err=nil	exists=false confirmed	PASS				
STG-04	TestJSONBatchSetGetBalance – BatchSet {alice:1000, bob:500, charlie:250}; GetBalance returns 3 accounts with correct values	02/25/2026	GetBalance() len=3; each value matches	3 accounts; values correct	PASS	Checkpoint write			
STG-05	TestJSONBatchCheckpointLogID – initial GetCheckpointLogID=0; SetCheckpointLogID(42); re-read returns 42	02/25/2026	initial=0; after set=42	Checkpoint ID tracked correctly	PASS	REQ-DATA-002 checkpoint			
STG-06	TestJSONBatchPersistence – write alice=999; checkpoint=10; close; reopen; values survive	02/25/2026	alice=999; exists=true; checkpoint.logID=10 after re-open	Data survives close/reopen	PASS	Durability			
STG-07	TestJSONBatchBatchOverwrites – pre-existing bob; removed; alice updated; charlie added by batch write	02/25/2026	bob: exists=false; alice=1000; charlie=750	Batch replaces entire dataset	PASS	Full snapshot semantics			
*** *** ***									
TEST TITLE		TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE			
Config Loader		https://github.com/SatyaPusha-256-Software-Corpus/Corpus/blob/main/Tests/ConfigLoader_test.go	Low	TC-CFG-001	CFG-UNIT-01	02/20/2026			
TEST DESCRIPTION		Unit tests for the configuration loader: valid JSON loading, default config values, validation rejection of zero shards, validation rejection of zero quorum, non-existent file error, and malformed JSON error.				TEST EXECUTED BY	ANAND DAS	EXECUTION DATE	02/20/2026
TEST DEPENDENCIES		internalconfig/config.go				TEST CONDITIONS	JSON config file written to temp dir per test	TEST CONTROL	Each test writes to own config or uses DefaultConfig(); no shared file state
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES	Column 1		
CFG-01	TestLoadConfigFromFile – write valid JSON with NumShards=4, ListenAddr=:5000; HeartbeatInterval=5; Load() parses correctly	02/20/2026	cfg.Cluster.NumShards=4; cfg.Coordinator.ListenAddr	All fields parsed correctly	PASS				
CFG-02	TestDefaultConfig – DefaultConfig() returns struct with NumShards=4, QuorumSize=4, no empty ListenAddr	02/20/2026	cfg.DefaultConfig() assertions pass	Default defaults present	PASS				
CFG-03	TestValidateConfigInvalid – DefaultConfig with NumShards=0; Validate() must return error	02/20/2026	Non-nil error from Validate()	Error returned for NumShards=0	PASS				
CFG-04	TestValidateConfigZeroQuorum – DefaultConfig with QuorumSize=0; Validate() must return error	02/20/2026	Non-nil error from Validate()	Error returned for QuorumSize=0	PASS				
CFG-05	Test.LoadFromFileNotExist – Load() (non-existent config path) must return error	02/20/2026	Non-nil error returned	Error returned for missing file	PASS				
CFG-06	Test.LoadFromFileBadJSON – Load() on file containing {invalid json} must return parse error	02/20/2026	Non-nil error returned	Error returned for bad JSON	PASS				
*** *** ***									
TEST TITLE		TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE			
Integration – Single Shard		https://github.com/SatyaPusha-256-Software-Corpus/Corpus/blob/main/Tests/IntegrationSingleShard_test.go	High	TC-INT-SG-001	INT-SG-01	03/05/2026			
TEST DESCRIPTION		Integration tests for single-shard transaction execution: happy-path transfer, overdraft rejection, conservation across multiple transfers, idempotency on retry, WAL-based crash recovery, and no-partial-state on abort.				TEST EXECUTED BY	ANAND DAS	EXECUTION DATE	03/05/2026
TEST DEPENDENCIES		shardserver_test.go, shardserver.go, shardserver_test.go, shardrecovery.go				TEST CONDITIONS	Temp WAL directory per test; ShardServer created with NewShardServer(); crash recovery tested by creating new server on same WAL path	TEST CONTROL	1 Cleanup closes ShardServer; temp directories auto-removed; no network required for single-shard tests
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES	Column 1		
SG-01	TestSingleShardTransferSuccess – alice=1000, bob=500; transfer 300; result=COMMITTED; alice=700, bob=800	03/05/2026	State=COMMITTED; alice=700; bob=800	COMMITTED; balances correct	PASS	REQ-TX-003			
SG-02	TestSingleShardInsufficientFundsAborts – alice=100; attempt transfer 500; result=ABORTED; state unchanged	03/05/2026	State=ABORTED; alice=100	ABORTED; no overdraft	PASS	REQ-DATA-002 no-negative invs			
SG-03	TestConservationOfMoneyAfterTransfer – 3 accounts total=3500; 4 sequential transfers; total must equal 3500 after each	03/05/2026	TotalBalance() = 3500 before and after all transfers	Conservation holds across 4 tons	PASS	REQ-DATA-002			
SG-04	TestIdempotency – transfer submitted twice; second submission returns same COMMITTED state; alice debited only once (1000)	03/05/2026	Both calls return COMMITTED; alice=900 (net 1000)	Idempotent; no double-debit	PASS	REQ-COORD-002 idemp. safety			
SG-05	TestCrashRecovery – commit 2 tons; close shard (crash sim); reopen with nil initial balances; alice=400, bob=400; total=800	03/05/2026	alice=400; bob=400; TotalBalance=800 after WAL	Full state recovered from WAL	PASS	REQ-DATA-003 Algorithm 5			
SG-06	TestAbortTransactionLeavesPartialState – alice=50; overdraft attempt 200; ABORTED; alice=50; bob=100 unchanged	03/05/2026	State=ABORTED; both balances unchanged	No partial state after abort	PASS	Atomicity guarantee			
*** *** ***									
TEST TITLE		TEST CODE LINK	PRIORITY	TEST CASE ID	TEST NUMBER	TEST DATE			
Integration – Cross Shard (2PC)		https://github.com/SatyaPusha-256-Software-Corpus/Corpus/blob/main/Tests/IntegrationCrossShard2PC_test.go	High	TC-INT-CS-001	INT-CS-01	03/15/2026			
TEST DESCRIPTION		Integration tests for cross-shard 2PC transactions over real HTTP: committed transfer, abort on insufficient funds, balance query endpoint, health endpoint, single-shard execute via HTTP and conservation across multiple cross-shard transactions.				TEST EXECUTED BY	ANAND DAS	EXECUTION DATE	03/15/2026
TEST DEPENDENCIES		coordinator/transaction.go, shardserver_test.go, message.go, coordinator/shardmap.go, coordinator/shardmap_test.go				TEST CONDITIONS	Two live HTTP shard servers on random ports; 2PC coordinator created in-process; 5 second HTTP timeout; shard servers started with startShard(HTTP)	TEST CONTROL	1 Cleanup shuts down HTTP servers; each test gets isolated shards on distinct free ports; no port reuse between tests
STEP ID	STEP DESCRIPTION	TEST DATE	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES	Column 1		
CS-01	TestCrossShardTransferSuccess2PC – allocateShard=1=1000; bobShard=2=500; transfer 300 via 2PC; result=COMMITTED; alice=700, bob=800	03/15/2026	State=COMMITTED; alice=700; bob=800	COMMITTED; correct balances on both shards	PASS	REQ-TX-003			
CS-02	TestCrossShardTransferInsufficientFundsAborts – alice=100, bob=500; attempt transfer 500 cross-shard; result=ABORTED; balances unchanged	03/15/2026	State=ABORTED; alice=100; bob=500	ABORTED; both shards unchanged	PASS	2PC global abort			
CS-03	TestHTTPHandlerBalance – allocateShard=1234; ShardClient GetBalance returns (1234, true, nil)	03/15/2026	bal=1234; exists=true; err=nil	Balance endpoint correct	PASS				
CS-04	TestHTTPHandlerHealth – GET /health on shard returns 200 with shard_id=shard-health	03/15/2026	HTTP 200; shard_id field correct	Health check passes	PASS	REQ-REP-003 liveness			
CS-05	TestHTTPHandlerExecuteSingleShard – allocate=1000, bob=500 on same shard; POST executes transfer 200; alice=800, bob=700	03/15/2026	State=COMMITTED; alice=800; bob=700	HTTP executes endpoint works	PASS	REQ-TX-001 API gateway path			

CS-06	TestConservationAcrossShards -- 4 accounts across 2 shards total=2500, 3 cross-shard txns, total unchanged			03/15/2026	TotalBalance across both shards = 2500 before any. Conservation holds cross-shard			PASS	REQ-DATA-002 global invariant																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
-------	--	--	--	------------	---	--	--	------	-------------------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--